AI redakční systém - specifikace pro AI agenta

Implementační zadání: architektura, datové modely, workflow, guardrails a milníky

Pracovní dokument • verze 1.0 • 11. srpna 2026

Cíl: vybudovat modulární AI redakční systém, který propojí data, SEO, analytiku, research, tvorbu obsahu a lidskou redakční kontrolu. Systém má zrychlit rozhodování, zvýšit organický výkon a umožnit globální škálování bez lineárního růstu počtu lidí.

0. Účel dokumentu

Tento dokument je implementační zadání pro coding/AI agenta. Agent má systém stavět modulárně, po malých ověřitelných krocích, s auditovatelnými datovými toky a bez automatického publikování v první fázi. Pokud konkrétní API, autentizace nebo datový formát nejsou v době implementace známé, agent je nesmí vymýšlet: má vytvořit adapter interface, mock a seznam chybějících credentials/rozhodnutí.

1. Primární cíle

- Denně vytvořit prioritizovaný editorial plan pro každý web/jazyk.
- Průběžně detekovat SEO opportunities z Search Console, SERP a interní historie.
- Rozhodovat assignment = AI | HUMAN | HYBRID podle transparentního scoringu.
- Generovat auditovatelný research package a následně draft.
- Provádět oddělený fact-check a quality gate.
- Připravovat WordPress payload; publish action držet za feature flagem a approval gate.
- Po publikaci měřit skutečný výkon a ukládat prediction-vs-actual pro budoucí kalibraci.

2. Architektonické principy

- Modular adapters: každá externí služba přes vlastní interface.
- Event + job model: dlouhé workflow rozdělit na idempotentní joby s retry.
- Structured outputs: mezi agenty používat validované JSON objekty, ne volný text.
- Human gates: schválení musí být explicitní stav workflow.
- Observability: logovat prompt/model/token cost/latency/source IDs/output schema version.
- Source provenance: každé faktické tvrzení v research package musí mít source reference.
- Least privilege: read-only credentials v pilotu; write scope pouze tam, kde je nutný.
- No silent failure: chybějící data musí snižovat confidence a být viditelná v UI/reportu.

3. Navržená struktura repozitáře

```
newsroom-ai/
  apps/
    api/
    dashboard/
    worker/
  packages/
    core/
    schemas/
    scoring/
    prompts/
    evaluation/
  adapters/
    wordpress/
    search_console/
```

```
  analytics/
  adsense/
  trends/
  serp/
  sources/
  social/
agents/
  editor_in_chief/
  research/
  seo/
  performance/
  writer/
  factcheck/
  publisher/
  evaluator/
workflows/
  daily_brief/
  topic_pipeline/
  content_refresh/
  post_publish/
storage/
  migrations/
  repositories/
tests/
  unit/
  integration/
  evals/
config/
docs/
```

4. Základní datové entity

Entita	Minimální pole
Site	id, domain, language, locale, wp_config_ref, brand_rules
Article	id, site_id, wp_id, url, title, author, published_at, content_type
TopicCandidate	id, site_id, topic, source_refs, freshness, scores, assignment, status
SearchMetric	date, site_id, url/query, impressions, clicks, ctr, position
TrafficMetric	date, article_id, users, sessions, pageviews, source
RevenueMetric	date, article_id/site_id, revenue, rpm
SerpSnapshot	query, locale, timestamp, positions, competitor_urls
ResearchPackage	topic_id, claims[], sources[], gaps[], confidence
Draft	topic_id, version, title_options, body, metadata, links, source_refs
Review	draft_id, checks[], severity, decision, reviewer_type
Experiment	target_id, change, hypothesis, baseline, result, window

5. Adapter contracts

Search Console adapter

Methods: list_sites(); query_performance(site, start, end, dimensions, filters); inspect_url(url) if available. Normalize to internal SearchMetric schema. Preserve original row and API timestamp for audit.

Analytics adapter

Methods: article_metrics(site, start, end); realtime/top_content where supported. Map URLs to canonical Article IDs.

AdSense/revenue adapter

Methods: revenue_by_site/date and revenue_by_url when source granularity permits. If URL-level revenue is unavailable, mark attribution_method and confidence; never fabricate article revenue.

WordPress adapter

Methods: fetch_posts, fetch_post, create_draft, update_draft, upload_media, set_taxonomy, publish. publish MUST require approval_token + feature flag in production.

6. Topic scoring

Implement configurable scoring, initially 0-100. Store both total and component scores. Suggested components: freshness 0-15, audience demand 0-20, SEO opportunity 0-20, brand relevance 0-15, expected value 0-15, source confidence 0-10, strategic value 0-10, minus competition/risk penalty 0-15. Weights must live in config/database, not hard-coded.

Assignment rules are a second classifier. HUMAN score rises with first-hand experience requirement, legal/reputational risk, opinion, product testing and originality requirement. AI score rises with structured facts, strong primary sources, urgency and repeatable format. HYBRID is default when both are material.

7. Daily brief workflow

- 01 ingest_internal_metrics: yesterday + rolling 7/28 day metrics.
- 02 ingest_search_console: query/page opportunities and anomalies.
- 03 ingest_trends_and_sources: candidate topics with timestamps and provenance.
- 04 deduplicate_topics: semantic clustering against today's queue and recent archive.
- 05 score_topics: store component scores + explanation.
- 06 assign_owner_type: AI/HUMAN/HYBRID + reason.
- 07 schedule_topics: proposed publication windows based on urgency and historical performance.
- 08 generate_daily_brief: ranked plan with confidence, expected outcome and required human action.

8. Topic-to-draft workflow

- Research agent collects primary sources first; secondary sources only as support.
- ResearchPackage must list claims, supporting source IDs, contradictions, unknowns and freshness.

- SEO agent adds target intent, query cluster, existing competing internal URLs, cannibalization risk, internal link candidates and update-vs-new recommendation.
- Editor-in-chief agent creates ContentBrief: angle, audience, required facts, prohibited claims, content type, assignment, length band and CTA/related links if relevant.
- Writer receives ContentBrief + ResearchPackage, not unrestricted raw browsing history.
- Fact-check agent independently validates material claims and outputs PASS / PASS_WITH_EDITS / BLOCK.
- Publisher prepares WP draft only after quality gate passes.

9. Structured output: TopicCandidate

```
{
  "topic": "string",
  "site_id": "string",
  "language": "string",
  "content_type": "news|guide|review|opinion|evergreen|update",
  "assignment": "AI|HUMAN|HYBRID",
  "scores": {
    "freshness": 0,
    "demand": 0,
    "seo_opportunity": 0,
    "brand_relevance": 0,
    "expected_value": 0,
    "source_confidence": 0,
    "strategic_value": 0,
    "risk_penalty": 0,
    "total": 0
  },
  "reasoning_summary": "short auditable explanation",
  "source_refs": ["source_id"],
  "suggested_publish_at": "ISO-8601|null",
  "confidence": 0.0
}
```

10. SEO opportunity detectors

- Striking distance: queries/pages typically positions 4-20 with meaningful impressions.
- CTR gap: compare observed CTR with internal expected CTR curve by position/device/country where enough data exists.
- Rising demand: acceleration in impressions/query demand over configurable windows.
- Decay: historically strong pages with sustained decline after seasonality adjustment where possible.
- Cannibalization: multiple internal URLs competing for same query cluster.
- Content gap: strategically relevant query clusters without adequate internal landing page.
- Refresh candidate: old article with backlinks/traffic/authority where update is cheaper than new URL.

11. Internal linking

Build a content graph from canonical URLs, topics/entities, existing links and performance. For each draft return ranked internal link suggestions with source URL, target URL, proposed anchor, contextual sentence location and confidence. Avoid sitewide mechanical anchors and excessive exact-match repetition. Track accepted/rejected suggestions to improve ranking.

12. Link-building support

Implement only opportunity detection and outreach preparation: linkable assets, broken/unlinked mentions where legally/technically obtainable, competitor link-gap research, outreach candidate lists and suggested personalized notes. Do not implement autonomous spam outreach, paid-link automation or uncontrolled account creation.

13. Fact-check contract

- Extract material factual claims from draft.
- For each claim map one or more source_refs.
- Prefer primary/official sources when available.
- Flag unsupported, stale, contradictory or overconfident claims.
- Return severity = info|warning|critical and proposed correction.
- BLOCK if critical claim lacks sufficient support or if draft implies first-hand experience that is not present.

14. Prompting rules

- Prompts are versioned files with explicit role, inputs, output schema and failure behavior.
- Do not mix research and final writing into one opaque prompt.
- Require citations/source IDs in research and fact-check outputs.
- Require uncertainty fields where source data is incomplete.
- Keep brand/style instructions separate from factual constraints.
- Never ask a model to invent analytics, rankings, quotes, product experience or source content.

15. Model routing and cost control

Create ModelRouter abstraction. Cheap/fast models: classification, deduplication, extraction, metadata, routine transformations. High-capability models: editorial prioritization, ambiguous research synthesis, difficult writing, final fact-check on important pieces. Record estimated and actual token/API cost per job, topic, article, site and day. Add daily/monthly budget guards.

16. Human approval state machine

```
DISCOVERED -> SCORED -> BRIEF_READY -> RESEARCHED -> DRAFTED
-> FACTCHECKED -> NEEDS_HUMAN_APPROVAL -> APPROVED
-> WP_DRAFT_CREATED -> PUBLISHED -> MEASURING

Failure branches:
BLOCKED_SOURCE | BLOCKED_FACTCHECK | REJECTED | RETRYABLE_ERROR
```

17. Post-publish measurement

- Capture indexation/visibility signals when available.
- Measure Search Console query/page metrics at configurable checkpoints.
- Measure Analytics traffic and revenue signals.
- Compare predicted topic score and expected outcome with actual performance.
- Store interventions (title change, update, internal links) as experiments rather than overwriting history.
- Do not let the system optimize solely for CTR; include quality, engagement, revenue and strategic value.

18. Evaluation suite

- Golden set of historical topics: would scoring have prioritized winners?
- Fact-check benchmark: unsupported claim detection.
- Duplicate/cannibalization benchmark.
- Internal-link relevance benchmark.
- Editorial assignment benchmark: AI vs HUMAN vs HYBRID.
- Cost/latency benchmark per workflow.
- Regression tests for every prompt/schema version.

19. Security and secrets

- Secrets only in environment/secret manager; never in prompts, repo or logs.
- Separate dev/staging/prod credentials.
- Pilot with read-only Google credentials.
- WordPress write credentials isolated to publisher service.
- Audit every external write operation.
- PII and user-level analytics should not be copied into model prompts unless strictly necessary and compliant.

20. Implementation milestones for coding agent

- M0: repository, config, schemas, database, job runner, logging, tests.
- M1: WordPress read adapter + archive ingestion.
- M2: Search Console + Analytics ingestion and normalized metrics.
- M3: opportunity detectors + scoring + daily brief, no generation.
- M4: research/SEO/editor brief pipeline with structured outputs.

- M5: writer + fact-check + review dashboard.
- M6: WordPress create/update draft behind approval gate.
- M7: post-publish measurement + experiment tracking.
- M8: selective autopilot for explicitly whitelisted low-risk content types.

21. Definition of done for each milestone

- Automated tests pass.
- Schema migrations included.
- All external calls have timeout/retry/rate-limit handling.
- Logs contain trace/job IDs and cost where applicable.
- No credential is committed.
- Failure mode is visible and recoverable.
- A short operator README explains setup and rollback.
- Demo can run against mock data before production credentials are required.

22. Initial instruction to the coding AI agent

```
Build this system incrementally. Start with M0 only.
Before coding, produce:
1) assumptions,
2) chosen stack and why,
3) repository tree,
4) database schema,
5) interfaces for external adapters,
6) risks / missing credentials,
7) acceptance tests.

Do not implement autonomous publishing.
Do not fabricate undocumented API capabilities.
Keep all model outputs schema-validated.
After M0 passes tests, stop and request approval for M1.
```